

VERTEX · מדריך תפעולי

VERTEX

מדריך התקנה מלא — סוכן AI אישי מקומי לשליטה במחשב

לקוח: Daniel Cohen — DC Trading X

שם הסוכן: Vertex | מילת הפעלה: "היי וורטקס"

מודל שפה מקומי מתמחה: ETHOVX

סביבת יעד: Windows 10/11, מחשב אישי בודד

גרסת מסמך: v1.0 | תאריך: 05 בספטמבר 2026

תוכן עניינים

01	1. מה זה Vertex — סקירה כללית
02	2. מה מוכן היום — סטטוס מלא ונבדק
03	3. התיקיה שדרכה הכול נכתב במחשב שלך
04	4. דרישות מקדימות
05	5. התקנה שלב-אחר-שלב
06	6. הגדרת מפתח NVIDIA NIM API
07	7. הפעלה ראשונה ובדיקת תקינות
08	8. הפעלה קולית — אימון "היי וורטקס"
09	9. Docker — הרצת קוד שנוצר ב-Sandbox
10	10. GPU + Ollama — אימון ETHOVX
11	11. שימוש יומיומי — דוגמאות פקודה
12	12. אבטחה ופרטיות — מה שמובטח בקוד
13	13. פתרון תקלות נפוצות
14	14. מה הלאה — Phase 4 ומעבר להפצה

1. מה זה Vertex — סקירה כללית

Vertex הוא סוכן AI אישי, פרטי ומקומי, שרץ על המחשב שלך בלבד (Windows 10/11) ומדבר איתך **אך ורק בעברית תקנית** — גם בטקסט וגם בקול. הוא בנוי לפי עיקרון יסוד אחד: שליטה מלאה שלך על כל פעולה רגישה. שום מחיקה, הרצת קוד, או פעולה הרסנית אחרת לא מתבצעת ללא אישור מפורש שלך בממשק.

נושא	פרטים
שם הסוכן	Vertex (וורטקס)
מילת הפעלה קולית	"היי וורטקס"
מודל שפה מקומי מתמחה	ETHOVX — שם קוד פנימי, רכיב נפרד
ספק מודלים ראשי	NVIDIA NIM API (מפתח קיים בידוך)
שפת תקשורת עם המשתמש	עברית בלבד — תמיד, ללא יוצא מן הכלל
סביבת הרצה	Windows 10/11, מחשב אישי בודד, ללא שרת מרוחק

מה Vertex יכול לעשות

ניהול ומחיקת קבצים (עם אישור), כתיבת קוד לפי בקשה, אוטומציית דפדפן גלויה עם דוח מסכם, בדיקת חוזקות אבטחה של המחשב, זיכרון קבוע הניתן למחיקה, וריצות אימון מתוזמנות למודל ETHOVX המתמחה בנושא שתבחר.

מה Vertex לא יעשה לעולם (בכוונה)

מחיקה "לחלוטין" ללא אשפה כברירת מחדל, שליטה אוטונומית בעכבר/מקלדת ללא Playwright מבוקר, קוד שמתקן ומריץ את עצמו ללא אישור, ו"למידה עצמאית 24/7" ללא מדידה אמיתית.

2. מה מוכן היום — סטטוס מלא ונבדק

המימוש בפועל (בריו, בנתיב 07_Development/Vertex/) עבר בדיקה אוטומטית מלאה: 16 מתוך 16 טסטים עוברים. הטבלה הבאה מפרטת בדיוק מה עובד ומוכן, ומה דורש ממך צעד נוסף על החומרה הפיזית שלך.

רכיב	סטטוס	הערה
Orchestrator (FastAPI + WebSocket)	מוכן	עולה, נבדק, מגיב על health/
Confirmation Gate (שער אישורים)	מוכן	שום פעולה הרסנית ללא אישור מפורש
Audit Log — Hash-Chain	מוכן	מזהה שיבוש בלוג, מוכח בטסט
ניהול קבצים + שחזור מהאשפה	מוכן	מחיקה=אשפה, שחזור אוטומטי דרך Windows Shell API
אוטומציית דפדפן + Tiling	מוכן	עד 4 משימות מקביליות, כל חלון ברביע מסך נפרד
Sandbox + קוד	מוכן	Docker sandbox אמיתי + סריקת bandit לפני אישור
ETHOVX — צינור אימון	מוכן	קורפוס מאושר, eval אמיתי, rollback אוטומטי
זיכרון קבוע (SQLite)	מוכן	עובדות, היסטוריית משימות, פקודת "תשכח"
בדיקת אבטחת מחשב	מוכן	Defender/Firewall/TPM/Secure Boot/פורטים
ממשק צ'אט מלא-מסך (Electron)	מוכן	RTL עברי, פאנל משימות, כרטיס אישור
קול עברי (TTS)	מוכן	Neural — Hila/Avri, איכות גבוהה
אימון wake-word "היי וורטקס"	דורש ממך	~20 דקות, הקלטת קול אישי (פרק 8)
Docker Desktop מותקן	דורש ממך	להפעלת sandbox בפועל (פרק 9)
GPU + Ollama לאימון ETHOVX	דורש ממך	חומרה + התקנה חד-פעמית (פרק 10)

שלוש השורות המסומנות "דורש ממך" אינן חוסרים בקוד — הן דורשות משהו שרק אתה יכול לספק (קול, חומרת GPU, התקנת תוכנת צד-שלישי). Vertex עובד במלואו דרך טקסט/כפתור מיקרופון גם בלעדיהן.

3. התיקייה שדרכה הכול נכתב במחשב שלך

מה	איפה בפועל
תיקיית ההתקנה (קוד התוכנה)	C:\Vertex (התקנת פיתוח) / C:\Program Files\Vertex installer (חתום)
נתוני משתמש (זיכרון, לוג ביקורת, קאש קול)	%APPDATA%\Vertex\
קובץ ההגדרות	C:\Vertex\config.yaml
מפתח NVIDIA NIM API	לא בקובץ בכלל — Windows Credential Manager מוצפן (DPAPI)
checkpoints של ETHOVX	%APPDATA%\Vertex\ethovx_checkpoints\

מקור הקוד

כל קוד המקור נמצא כרגע בריפו הפרויקט תחת 07_Development/Vertex/. זו תיקיית ה-source — ממנה מריצים את סקריפט ההתקנה שמעתיק את כל הקבצים בפועל ל- C:\Vertex על המחשב שלך.

4. דרישות מקדימות

יש להתקין פעם אחת, לפני הכול, על מחשב ה-Windows 10/11 שלך:

- 1 Python 3.12 — מ- `python.org/downloads` . חשוב לסמן "Add python.exe to PATH" בזמן ההתקנה.
- 2 Node.js 20 LTS ומעלה — מ- `nodejs.org` . נדרש לממשק המשתמש (Electron).
- 3 Git (אופציונלי) — מ- `git-scm.com` , רק אם תרצה למשוך עדכוני קוד בעתיד.
- 4 מפתח NVIDIA NIM API פעיל — כבר קיים בידיך.

רכיב חומרה	מינימום	מומלץ (עם ETHOVX)
CPU	4 ליבות	+8 ליבות
RAM	16GB	32GB
GPU	לא נדרש (ETHOVX לא ירוץ)	NVIDIA RTX עם 8GB+ VRAM
אחסון	5GB (תוכנה בלבד)	+30GB (כולל קורפוס/checkpoints)
מיקרופון	כל מיקרופון מוכר ע"י Windows	עם ביטול רעש רקע

5. התקנה שלב-אחר-שלב

דרך א' — סקריפט ההתקנה המהיר (מומלץ)

- 1 העתק/הורד את תיקיית `07_Development/Vertex` מהריפו למחשב ה-Windows שלך (למשל לשולחן העבודה, כמיקום זמני).
- 2 פתח **PowerShell** רגיל (לא חובה כ-Administrator) בתוך תת-התיקייה `scripts`.
- 3 הרץ את שתי הפקודות הבאות:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass  
.\install_windows.ps1
```

- 4 הסקריפט מבצע אוטומטית: יצירת `C:\Vertex`, בניית סביבת Python (`venv`) והתקנת כל התלויות, התקנת תלויות הממשק (`npm install`), יצירת `config.yaml`, יצירת קיצור דרך "Vertex" על שולחן העבודה, ובדיקת זמינות Docker/Ollama (עם דיווח ברור אם הם חסרים).

דרך ב' — Installer חתום דיגיטלית (להפצה סופית)

בתיקיית `installer/build_installer.iss` נמצא סקריפט **Inno Setup** מלא. זו הדרך המקצועית לפרודקשן: `build` עם `PyInstaller + electron-builder`, חתימה דיגיטלית ל-`exe` (`signtool`), והתקנה בלחיצה כפולה על `VertexSetup.exe` — בדיוק כמו כל תוכנת Windows רגילה, ללא PowerShell בצד המשתמש הסופי. מתאים לשלב שבו הקוד יציב ומוכן להפצה קבועה.

6. הגדרת מפתח NVIDIA NIM API

חשוב

המפתח לעולם לא נשמר בקובץ טקסט גלוי. שתי דרכים אפשריות:

דרך מהירה (פיתוח/בדיקה) — משתנה סביבה

```
$env:VERTEX_NIM_API_KEY = "המפתח-שלך-כאן"
```

זמני בלבד — נעלם בסגירת ה-PowerShell אלא אם תגדיר אותו כמשתנה סביבה קבוע.

דרך מומלצת (פרודקשן) — Windows Credential Manager מוצפן

```
cd C:\Vertex
.\venv\Scripts\python.exe -c "from core.config import save_secret_windows,
CREDENTIAL_TARGET; save_secret_windows(CREDENTIAL_TARGET, input('NIM:
הדבק כאן את מפתח ה-'))"
```

מרגע זה Vertex טוען את המפתח אוטומטית בכל הפעלה — מוצפן, ולא בלוג ולא ב- `config.yaml` אף פעם.

7. הפעלה ראשונה ובדיקת תקינות

- 1 לחיצה כפולה על קיצור הדרך "Vertex" על שולחן העבודה, או הרצת `C:\Vertex\scripts\start_vertex.bat`.
- 2 ייפתחו שני תהליכים: ה-Orchestrator (שרת מקומי על `127.0.0.1:8420` בלבד — אין גישה מרחוק לעולם) וחלון הצ'אט המלא-מסך.
- 3 בדיקת תקינות מהירה: כתוב "שלום Vertex" בשדה ההודעה. אמור לחזור מענה בעברית מלאה, ותשמע גם קול (אם עדכנת את המפתח).

הפעלה חוזרת

לא פותחת שוב את תיקיית ההתקנה — רק את חלון הצ'אט. סגירת החלון ממזערת אותו לשורת המשימות (tray); Vertex ממשיך להאזין ברקע.

8. הפעלה קולית — אימון "היי וורטקס"

מילת ההפעלה מותאמת אישית לקול שלך (לא מודל גנרי) כדי למנוע הפעלות שווא ולשמור על פרטיות מלאה — שום אודיו לא נשלח לרשת לפני זיהוי המילה. תהליך חד-פעמי של כ-20 דקות:

1 הקלטת 40–60 דוגמאות חיוביות — אתה אומר "היי וורטקס" בגוונים/מרחקים שונים:

```
cd C:\Vertex
.\venv\Scripts\python.exe wake_service\record_samples.py --label positive --count 50
```

2 הקלטת 200+ דוגמאות שליליות (דיבור רגיל, טלוויזיה, מוזיקה):

```
.\venv\Scripts\python.exe wake_service\record_samples.py --label negative --count 200
```

3 אימון מודל openWakeWord קל-משקל (ONNX) — פקודות מדויקות ב- `wake_service/README.md`.

4 הרץ שוב את `install_windows.ps1` — הוא יזהה את המודל המאומן וירשום את שירות ההאזנה כמשימה מתוזמנת אוטומטית בכניסה למשתמש.

אין חובה

עד לאימון המודל אפשר להשתמש ב-Vertex במלואו דרך טקסט וכפתור המיקרופון בממשק — ה-wake-word הוא נוחות נוספת, לא תלות.

9. Docker — הרצת קוד שנוצר ב-Sandbox

כדי ש-Vertex יוכל להריץ בפועל (ולא רק לכתוב) קוד שהוא מייצר עבורך — בבידוד מלא, ללא גישה רשת וללא גישה לדיסק החי — נדרש Docker Desktop:

1 הורד והתקן מ- `docker.com/products/docker-desktop`.

2 הרץ את Docker Desktop פעם אחת כדי שהשירות יעלה ברקע.

3 זהו. בפעם הראשונה שתבקש מ-Vertex "תכתוב לי סקריפט ל-X", המערכת תמשוך אוטומטית את תמונת `python:3.12-slim` ותריץ בתוכה את הקוד עם `--network none` ומגבלות CPU/זיכרון, לפני שהיא מבקשת את אישורך.

בלי Docker

Code-Gen עדיין יכתוב קוד ויציא סריקת bandit סטטית, אבל יסרב לאשר הרצה דינמית ויגיד לך בפירוש "Docker אינו מותקן" — לא ידמה בדיקה שלא קרתה.

10. GPU + Ollama — אימון ETHOVX

ETHOVX הוא מודל שפה מקומי קטן שאפשר לאמן על נושא שאתה בוחר (למשל תיעוד קוד Pine Script, תמיכה טכנית בנושא מסוים). ריצת אימון אמיתית דורשת:

1 כרטיס מסך NVIDIA עם 8GB+ VRAM פנוי (ר' טבלת חומרה בפרק 4).

2 התקנת Ollama מ- `ollama.com/download`, ואז משיכת המודל הבסיסי:

```
ollama pull llama3.2:3b
```

3 עריכת `config.yaml` והוספת נתיבי הקורפוס המאושרים לאימון (רק מה שאתה מאשר — Vertex לעולם לא סורק את כל האינטרנט):

```
ethovx:  
  base_model: "llama3.2:3b"  
  approved_sources: ["C:\\Vertex\\corpus\\my_topic"]
```

4 אמור/כתוב ל-Vertex: "תתחיל אימון ETHOVX". הוא יבקש אישור מפורש, ולאחריו ירוץ ברקע (לא חוסם את הצ'אט) וידווח דוח מסכם עם דיוק לפני/אחרי, מבוסס על 5 שאלות בדיקה קבועות — לא אחוז שרירותי.

בטיחות מובנית

אם ריצת אימון חדשה מורידה את הדיוק בפועל — מתבצע **rollback אוטומטי** לצ'קפוינט הקודם. נשמרים תמיד 3 הצ'קפוינטים האחרונים בלבד.

11. שימוש יומיומי — דוגמאות פקודה

מה שאתה אומר/כותב	מה Vertex עושה
"וורטקס, תמחק את קבצי ה-tmp בהורדות"	סורק, מציג כרטיס אישור עם מספר קבצים+גודל, ורק בלחיצה על "אשר" מעביר לאשפה
"וורטקס, תפתח דפדפן ותחפש X"	שואל באיזה דפדפן, פותח חלון גלוי, מדווח שלבים בזמן אמת + דוח מסכם בסיום
"וורטקס, תבדוק לי אבטחה"	סורק Defender/Firewall/TPM/Secure Boot/פורטים, מציג ציון סיכון 0–100 + המלצות
"וורטקס, תכתוב לי סקריפט ל-X"	כותב קוד, מריץ ב-Docker sandbox, מציג diff+תוצאות, ומבקש אישור לפני שמירה
"וורטקס, תשכח את X"	מוחק את הרשומה התואמת מהזיכרון הקבוע — מייד, בלי אישור נוסף
"וורטקס, בטל את הפעולה האחרונה"	משחזר קבצים מהאשפה בפועל (Windows Shell API) אם ניתן, ומדווח מה שוחזר
"וורטקס, תתחיל אימון ETHOVX"	מבקש אישור, מריץ job מתוזמן ברקע, מדווח דוח דיוק לפני/אחרי בסיום

12. אבטחה ופרטיות — מה שמובטח בקוד

- **Confirmation Gate**: שום מחיקה/הרצת קוד/שינוי — ללא אישור מפורש. אין תגובה = דחייה (ברירת מחדל בטוחה).
- **Audit Log עם Hash-Chain**: כל פעולה נרשמת; שיבוש בלוג מזוהה מיידית.
- **הגנת Prompt Injection**: תוכן דף/קובץ חיצוני מסומן כ"נתון בלבד" ולעולם לא כפקודה — גם אם הוא מנסה "להתחזות" להוראה.
- **רשת**: Bind רק ל- 127.0.0.1 — אין גישה מרחוק לעולם; תעבורה יוצאת רק ל-NVIDIA NIM ולדפדפן המבוקר.
- **הרשאות**: Vertex רץ כמשתמש רגיל (לא Administrator) כברירת מחדל.
- **מפתח ה-API**: מוצפן ב-Windows Credential Manager (DPAPI) — לעולם לא בטקסט גלוי.
- **עברית תמיד**: אכיפה בקוד (לא רק בפרומפט) בשלוש שכבות עצמאיות — TTS עברי, הוראת-ברזל אוטומטית ל-system prompt, וגם בתרחיש הגנת ה-injection.

13. פתרון תקלות נפוצות

פתרון	תופעה
ודא שה-Orchestrator רץ (חלון PowerShell ממוזער ברקע); בדוק <code>http://127.0.0.1:8420/health</code> בדפדפן	חלון הצ'אט נפתח אך לא מגיב
חזור לפרק 6 והגדר את המפתח דרך Credential Manager	"לא הוגדר מפתח "NVIDIA NIM API"
ודא חיבור אינטרנט פעיל (edge-tts דורש רשת); בדוק שה- <code>tts_voice_gender</code> ב- <code>config.yaml</code> תקין	אין קול כלל
ודא שהמודל אומן (פרק 8) והרגישות (<code>wake_word_threshold</code>) לא גבוהה מדי	"היי וורטקס" לא מזוהה
התקן והפעל Docker Desktop (פרק 9); בדוק עם <code>docker --version</code> ב-PowerShell	"Docker אינו מותקן" בבקשת קוד
ודא GPU+Ollama מותקנים (פרק 10); בלי <code>torch/peft/transformers</code> המערכת מדווחת זאת בפירוש ולא מדמה הצלחה	אימון ETHOVX לא בוצע בפועל
ודא ש- <code>winshell</code> מותקן (חלק מ-requirements.txt ב-Windows); אחרת שחזר ידנית מהאשפה של Windows	שחזור אוטומטי מהאשפה לא עבד

14. מה הלאה — Phase 4 ומעבר להפצה

Phase 1 (ליבה), Phase 2 (הרחבת יכולות) ו-Phase 3 (ETHOVX) — שלמים, נבדקים, ומוכנים להרצה על המחשב שלך כפי שמתואר במדריך זה. השלב היחיד שנותר לפי מפת הדרכים המקורית הוא **Phase 4**: אפליקציית טלפון כלקוח נוסף המתחבר לאותו Orchestrator — אינו בהיקף המסירה הנוכחית, ונשמר לעתיד לפי הצורך.

למעבר מהתקנת פיתוח (`install_windows.ps1`) להפצה קבועה עם installer חתום דיגיטלית — ר' פרק 5, "דרך ב'", ו- `installer/build_installer.iss` בקוד המקור.